



Figure 5: Consolidated pictorial representation for Code-1

```

signed char x = 0xFF;
0040426: c6 44 24 1e ff      movb $0xff,0x1e(%esp)
unsigned char y = 0xFF;
004042b: c6 44 24 1f ff      movb $0xff,0x1f(%esp)

//Compare both the values of x and y
if (x == y)
0040430: 0f be 54 24 1e      movbl 0x1e(%esp),%edx
0040435: 0f be 44 24 1f      movbl 0x1f(%esp),%eax
004043a: 39 c2              cmp     %eax,%edx
004043c: 75 0e              jne     0x40441c <main+0x2f>
{
004043e: c7 04 24 f9 04 00   movl $0x80404f9,(%esp)
0040445: e8 a6 fe ff ff      call 0x4042f0 <puts@plt>
004044a: eb 0c              jmp     0x404450 <main+0x3b>
}
else
{
004044e: c7 04 24 f6 04 00   movl $0x80404f6,(%esp)
0040453: e8 98 fe ff ff      call 0x4042f0 <puts@plt>
}
return 0;

```

Figure 6: Disassembled code for the source Code-1

```

Output
Code
[Satya]~/Article-IntegralPromotion/Codes
gcc test_3.c
[Satya]~/Article-IntegralPromotion/Codes
./a.out
Res: 75

```

Figure 7: Output of Code-3



Note: In C99, integer promotion is clearly defined in the following rule (6.3.1.1): “If an int can represent all values of the original type, the value is converted to an int; otherwise, it is converted to an unsigned int. These are called the integer promotions. All other types are unchanged by the integer promotions.”

Let us consider another example (Code-3) to understand integer promotion:

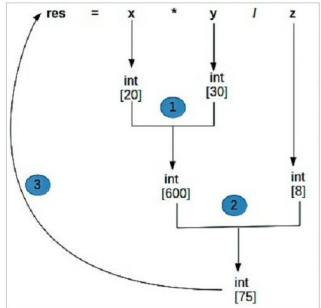
```

1 #include <stdio.h>
2 int main()
3 {
4     signed char x, y, z, res;
5     x = 20;
6     y = 30;
7     z = 8;
8     res = x * y / z;
9     printf("Res: %d\n", res);
10    return 0;
11 }

```

Code-3: Code to show how integer promotion takes place in an expression that involves arithmetic operators

Consider the expression `res = x * y / z`; present in Code-3. In this expression, there are three operators (`=`,

Figure 8: Pictorial representation of the type promotion for the expression `res = x * y / z`

```

int main()
{
0040410: 55                push    %ebp
0040411: 89 e5             mov     %esp,%ebp
0040420: 83 e4 f0          and     $0xfffff0,%esp
0040423: 81 ee 20          sub     $0x20,%esp
    signed char x, y, z, res;
    x = 20;
0040426: c6 44 24 1c 14    movb    $0x14,0x1c(%esp)
    y = 30;
004042b: c6 44 24 1d 1a    movb    $0x1a,0x1d(%esp)
    z = 8;
0040430: c6 44 24 1e 08    movb    $0x8,0x1e(%esp)
    res = x * y / z;
0040435: 0f be 54 24 1c    movsbl  0x1c(%esp),%edx
004043a: 0f be 44 24 1d    movsbl  0x1d(%esp),%eax
004043e: 0f ae c2          imul    %edx,%eax
0040442: 0f be 4c 24 1e    movsbl  0x1e(%esp),%ecx
0040447: 99               cltd
0040448: f7 f9           idiv    %ecx
004044a: e8 a4 24 1f      mov     %eax,0x1f(%esp)
    printf("Res: %d\n", res);
004044e: 0f be 44 24 1f    movsbl  0x1f(%esp),%eax
0040453: 89 44 24 04       mov     %eax,0x4(%esp)
0040457: c7 04 24 00 85 04 movl    $0x8040485,4(%esp)
004045e: e8 8d fe ff ff    call    0x4042f0 <printf@plt>
    return 0;
0040463: b8 00 00 00 00    mov     $0x0,%eax
}

```

Figure 9: Disassembled code for the source Code-3

`*`, `/`). Among these three, the `*` and `/` operators have the same precedence; hence, associativity comes into the picture. According to the associativity rule for the arithmetic operators, evaluation is from left to right. So, `*` will be evaluated first. Before evaluating the sub-expression `x * y`, both the operands will be type promoted to either `int` or `unsigned int`, according to the rules of c99 standards (see the Note above).

The result of `x * y` is 600 according to the values given in Code-3, which seems to be arithmetic overflow because 600 does not fall in the range -128 to +127 (minimum and maximum limit of the `signed char`). However, due to integer promotion the operation is smoothly carried out without any overflow.

In the expression, the result of the sub-expression is further divided by `z`, which is a `signed char`. As a result, 600, which